COMP 3300 – Fall 2025
Assignment 1, 10 pts.
Due: Sunday, Aug 24th @11:55pm

Objectives

- Become familiar with the Visual Studio IDE
- Start to learn programming in C#

Notes

- Please read through this entire document before you begin, so you know all the requirements for the program at the outset.
- **Any program that does not compile will receive a zero (0). Partial credit is not possible for any program that does not compile.**

Getting Started

1. Start Visual Studio 2022 and create a new `C# Console Application` that is a .NET 8 application, i.e., it can run on Windows, Linux, and MacOS. This should **NOT** be a .NET Framework application.

    a. Name the project *`FirstnameLastname`*`A1`, e.g., `SallieMaeA1`.

    b. Create the application with a `Main` method.

        - To do this make sure to check Do not use top-level statements.

2. Build and run the application to verify it runs. It should display `Hello, World` in the console application window that is created upon running the application.
3. Make sure that each method created for this assignment has the proper XML documentation present.

Requirements

Introducing a Driver Method

1. The `Main` method should not have any functionality in it, other than starting the "driver" method of the class that the main method is contained within. To do this, do the following:

    a. Add an instance method (non-static method) to the class named `Run`.

    b. The `Run` method should return void and have no parameters.

2. In the `Run` method have it output the following:

    `*First Name* *LastName*` `Assignment 1`

    Example:

    ```
    Joseph Martinez Assignment 1
    ```

3. In the `Main` method, do the following:

    a. Delete the line that outputs `Hello, World`.

    b. Create an instance of your class object.

    c. Using the class object, call the `Run` method.

4. Build and run the application to very it runs and displays *`Firstname Lastname`* `Assignment 1.`

Prompting user for two words

1. Add two data members also known as instance variables or fields to the class to store the two words.

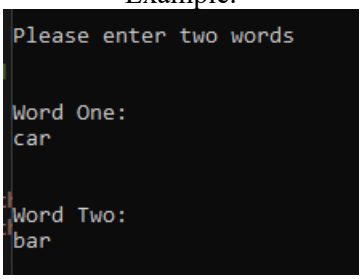
2. Add a method that will prompt the user and read-in two words from the user. Hint: The `Console::ReadLine` method can read in user input from the console.

    a. The words should be converted to lowercase after being read in.

    b. The words read in must be stored in the associated data members (fields) of the class.

3. Call the method from your `Run` method and then verify the new functionality works correctly.

Example:



Word fun output

1. Add a method that will display each word, its length, and the number of characters longer the longer word is compared to the shorter word. The example output below should help clarify.

Example 1
Assuming words entered by user are *car* and *tar*

```
You have entered the following two words: car and tar


--------------------------------


Word One: car; Length 3
Word One: tar; Length 3
car is the same number of letters as tar
```

Example 2
Assuming words entered by user are *Baseball* and *horn*

```
You have entered the following two words: baseball and horn


--------------------------------


Word One: baseball; Length 8
Word One: horn; Length 4
baseball has 4 more characters than horn
```

Example
Assuming words entered by user are *pear* and *orange*

```
You have entered the following two words: horn and automobile


------------------------------


Word One: horn; Length 4
Word One: automobile; Length 10
horn has 6 fewer characters than automobile
```

a. In the `Run` method call the method and then verify the functionality is correct.

2. Add a method that will display a word square for a word that is passed as a parameter to the method. A word square is printing out the word the number of times the length of the word is. E.g., if the word `apple` were passed to the method the method would print out `apple` 5 times, if `cat` were passed to the method `cat` it would be printed out 3 times.

   Example output for *apple*:

```
apple
apple
apple
apple
apple
```

   a. In the `Run` method call this method for each of the two words entered and verify the output is correct.

3. Add a method that will display a word pyramid (see below) for a word that is passed as a parameter to the method. E.g., if the word `apple` were passed to the method the method would output the following:

```
a
ap
app
appl
apple
appl
app
ap
a
```

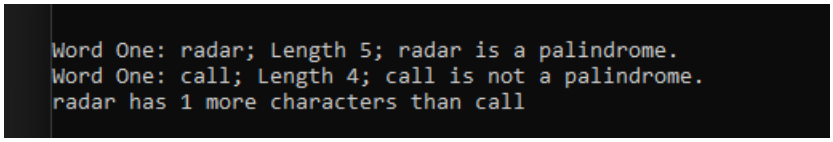
   a. In the `Run` method call the method for each of the two words entered and verify the output is correct.

4. Add a method that accepts a string as a parameter and returns a `bool` to indicate whether a string is a palindrome or not. A palindrome is word that reads the same backwards as it does forward, e.g. `radar` is a palindrome.

    a. Hint: For this method it may be beneficial to create a word that is the reverse of the word passed in. I suggest Googling how to reverse a string in C#. There are several ways to do this.

5. Modify the initial output about each word so that after the length it displays whether the word is a palindrome or not.

    Assuming words entered by user are *`radar`* and *`call;`* The initial output would be:



    a. After adding the functionality verify it works correctly.

Example output of all functionalities to this point is below:

```
Joseph Martinez Assignment 1


Please enter two words


Word One:
racecar


Word Two:
ball


You have entered the following two words: racecar and ball


--------------------------------


Word One: racecar; Length 7; racecar is a palindrome.
Word One: ball; Length 4; ball is not a palindrome.
racecar has 3 more characters than ball


racecar
racecar
racecar
racecar
racecar
racecar
racecar


ball
ball
ball
ball


r
ra
rac
race
racec
raceca
racecar
raceca
racec
race
rac
ra
r


b
ba
bal
ball
bal
ba
b
```

Looping until user enters quit.

1. Modify the functionality so that after it displays the output for the two words entered it would prompt the user again for two more words and it will continue to do this until the user enters `quit`, case insensitive, for one of the words at which point it displays `Goodbye` and exits.
    - What you say/mention to the user is up to you, but it should be clear to them that you MUST be able to type quit, case agnostic, to end the program. Anything else typed should not end the program.

2. After adding the functionality verify it works correctly.

Grading breakdown

- 10 pts. – Exercise completed successfully with at most only a minor error and follows best programming practices.
- 7 pts. – Exercise is missing some functionality or has some errors in it and mostly follows best programming practices.
- 4 pts. – Exercise is missing most of the major functionality or has major errors or best programming practices not followed.
- 0 pts. – Exercise not submitted, does not compile, or barely started.

Submission

Make sure you clean your solution (Build → Clean Solution) and then zip up your project which needs to include the solution file into a ZIP file called *YourName*A1.zip and submit the assignment by the due date.